

SYSTEM DESIGN BASICS

Module 3 • Learning Units 3.1–3.8

Quick-study guide

Eight focused pages. Each page explains one Learning Unit in plain language, with the key ideas, a practical example, common mistakes, and an exam-ready answer.

How to use this guide

- Read the short explanation first. Make sure you understand the idea before memorising terms.
- Use the example to connect the idea to a real product.
- Cover the exam answer and try explaining the topic in your own words.
- Use the memory line for a quick revision before a test.

The design habit behind the module

Start with the user's problem. Decide what the system must do. Give each responsibility a clear home. Then show how the parts fit together and how each important part works inside.

Contents

- LU 3.1 Understanding the Problem
- LU 3.2 Designing User Experiences
- LU 3.3 Defining Product Requirements
- LU 3.4 Design Foundations
- LU 3.5 Designing the System
- LU 3.6 Visualizing the Architecture
- LU 3.7 Organizing the Components
- LU 3.8 Detailing the Components

Understanding the Problem

Start with the need, not the technology.

What it means

System design begins by understanding the real problem a product must solve. Before choosing components or drawing boxes, identify who has the problem, what they are trying to achieve, and what a successful outcome looks like.

Why it matters

If the problem is misunderstood, even a technically impressive system can solve the wrong thing. Clear problem understanding prevents wasted work and gives every later design decision a reason.

Key ideas to remember

- **Users:** the people who interact with or are affected by the system.
- **Goals:** what those people want to accomplish.
- **Requirements:** what the system must do or achieve to support those goals.
- **Constraints and assumptions:** limits we know about, and things we are temporarily treating as true.

Example

For a campus carpool app, the problem is not “we need a database.” The problem is that students need a convenient way to find and coordinate rides to campus.

Common mistake

Jumping straight to tools, databases, or architecture before stating the user problem. A technology choice is not the problem statement.

Exam-ready answer

Understanding the problem means identifying users, their goals, the needs the system must satisfy, and the constraints around the solution. It provides the foundation for requirements and architecture.

Remember: Who has the problem? What do they need? What limits the solution?

Designing User Experiences

Plan what the user experiences before building it.

What it means

User experience (UX) is the overall experience a person has while trying to achieve a goal. User interface (UI) is the visible and interactive part they use, such as screens, buttons, forms, and menus. A user journey describes the steps a person takes to complete a task. A wireframe is a simple layout sketch of a screen.

Why it matters

A system can work internally and still be frustrating to use. Planning the journey helps reveal missing steps, confusing choices, and information the user needs at each point.

Key ideas to remember

- **UX:** the whole experience and how easy or difficult the task feels.
- **UI:** the controls and visual layout through which the user interacts.
- **User journey:** the ordered steps from the user's starting point to their goal.
- **Wireframe:** a rough screen plan focused on structure, not decoration.

Example

In a library-booking app, a journey might be: open the app → search for a book → choose a copy → reserve it → see confirmation. A wireframe shows where search, results, and confirmation appear.

Common mistake

Treating UI and UX as the same thing, or polishing colours before checking whether the user journey makes sense.

Exam-ready answer

UX concerns the user's overall experience, while UI is the interface used to interact with the system. User journeys map task steps, and wireframes sketch screen structure before detailed implementation.

Remember: UX = experience, UI = controls, journey = steps, wireframe = rough screen.

Defining Product Requirements

Turn user needs into clear statements of what the product must do.

What it means

Product requirements describe what a product needs to provide. A Product Requirements Document (PRD) records the problem, intended users, goals, scope, and requirements so the team shares the same understanding before building.

Why it matters

Without clear requirements, different people may build different versions of the product in their heads. Requirements help the team decide what is in scope, what success means, and what needs to be tested.

Key ideas to remember

- **Functional requirements:** behaviours or capabilities the system must provide.
- **Scope:** what the product will and will not cover.
- **Clear wording:** state the expected behaviour so it can be checked.
- **User-goal link:** each requirement should help a user or stakeholder achieve a real goal.

Example

Vague: "The app should have good search." Clearer: "A student can search available study rooms by date and time and view matching rooms."

Common mistake

Writing implementation choices as requirements. "Use PostgreSQL" says how the team might build something, not what the user needs the product to do.

Exam-ready answer

A PRD documents the product's purpose, users, scope, and requirements. Functional requirements specify the actions or behaviours the system must support and should be clear enough to validate.

Remember: Need → requirement → something the team can verify.

Design Foundations

Make the design clear, changeable, and realistic.

What it means

Good design divides a system into understandable responsibilities. Modularity separates a system into parts; abstraction hides unnecessary detail; separation of concerns keeps different responsibilities apart. Low coupling means parts depend less on one another. High cohesion means the work inside a part belongs together.

Why it matters

Software changes. A clear design makes it easier to understand the system, update one responsibility, and avoid accidental damage elsewhere.

Key ideas to remember

- **Non-functional requirements (NFRs):** qualities or limits such as performance, reliability, usability, or security.
- **Modularity:** divide a system into meaningful parts.
- **Abstraction:** show what matters while hiding internal detail.
- **Low coupling:** reduce unnecessary dependencies between parts.
- **High cohesion:** keep closely related work together.
- **Constraints / assumptions:** known limits / temporary beliefs used for planning.

Example

If notifications are handled by a separate module, changing reminder wording should not require rewriting room booking logic. The responsibilities are separated.

Common mistake

Thinking modularity means making as many tiny parts as possible. The goal is useful separation, not a box for every tiny action.

Exam-ready answer

Design foundations help create systems that are understandable and maintainable. They include NFRs, modularity, abstraction, separation of concerns, low coupling, high cohesion, constraints, and assumptions.

Remember: Keep related work together; keep unrelated work apart.

Designing the System

Turn requirements into major system parts.

What it means

High-Level Design (HLD) describes the main structure of a solution. It identifies the major components or modules, their responsibilities, and the important ways they interact. It does not describe every internal coding detail.

Why it matters

Requirements tell us what the system must achieve. HLD helps decide which major parts should be responsible for achieving it and how those parts fit together.

Key ideas to remember

- **Decomposition:** break a large problem into smaller responsibilities.
- **Major modules:** meaningful parts of the system, each with a clear job.
- **Responsibility:** state what each module owns or does.
- **Interaction:** identify which parts need information or services from other parts.

Example

For an event-booking app, HLD might include Event Listings, Booking, Payments, and Notifications. Booking manages reservations; Notifications sends confirmations. These are responsibilities, not just names.

Common mistake

Making one giant module responsible for everything, or splitting the system into modules without explaining why each exists.

Exam-ready answer

High-Level Design translates requirements into a system structure by decomposing the system into major modules, assigning responsibilities, and describing their interactions.

Remember: Requirements say what is needed; HLD decides the major parts responsible for it.

Visualizing the Architecture

Draw the system so people can understand its structure.

What it means

An architecture diagram is a visual representation of the system's major parts and how they relate. Depending on its purpose, it may show users, frontend, backend, external services, and data storage. The diagram should make structure and important connections easy to follow.

Why it matters

A diagram gives the team a shared picture of the design. It can expose missing parts, unclear boundaries, and connections that need explanation before implementation begins.

Key ideas to remember

- **System boundary:** what belongs to the system and what is outside it.
- **Components / layers:** the major parts that have different responsibilities.
- **Connections:** how parts communicate or depend on one another.
- **Labels:** name parts and connections clearly; avoid unexplained arrows.

Example

A basic web app diagram: User → Browser/Frontend → Backend → Database. The arrows indicate communication; the boxes indicate responsibilities or system parts.

Common mistake

Drawing decorative boxes without showing meaningful relationships, or adding so much detail that the big picture disappears.

Exam-ready answer

Architecture diagrams visualize the major components of a system and their relationships. They help stakeholders understand system structure, boundaries, and interactions.

Remember: Boxes show the parts; arrows show how the parts connect.

Organizing the Components

Split the backend into parts with one clear responsibility.

What it means

A component is a meaningful part of a system with a clear responsibility. Organizing components means grouping related work together and separating unrelated work. A UML component diagram shows backend components and how they depend on one another.

Why it matters

Clear components are easier to understand, assign ownership to, test, and change safely. A component doing everything becomes a giant blob; a vague “Helpers” component explains almost nothing.

Key ideas to remember

- **One responsibility per component:** explain each component's job in one sentence.
- **Provided interface (lollipop):** a service the component offers to others.
- **Required interface (socket):** a service the component needs from another component.
- **Dependency:** one component uses another. A dashed dependency arrow can also show this.
- **Architecture vs component diagram:** architecture shows the larger running system; component diagram zooms into a part, such as the backend.

Example

Skill Swap can have User Profiles, Skill Search, Matching, Session Scheduling, Reviews, and Notifications. Skill Search uses profile information; Session Scheduling may use Matching and Notifications.

Common mistake

Creating one mega-component, using vague catch-all names, or assuming a component diagram shows the time order of a user journey. It shows structure and dependencies.

Exam-ready answer

A component diagram represents the meaningful components of a system and the dependencies between them. Components should have clear responsibilities, with provided and required interfaces showing services offered and needed.

Remember: Good split = related work together, unrelated work apart.

Detailing the Components

Explain how one component works inside.

What it means

Low-Level Design (LLD) describes the internal workings of one component clearly enough that a developer can build it without guessing. It goes deeper than a component diagram: it explains internal parts, operations, data, main flow, and failure behaviour.

Why it matters

A box labelled “books sessions” is not enough to implement booking. The team must decide what input is accepted, what is stored, what happens if a slot is taken, and what to return when something fails.

Key ideas to remember

- **Typical design path:** PRD → HLD → Architecture Diagram → Component Diagram → LLD.
- **Seven LLD sections:** (1) overview and scope, (2) internal structure, (3) operation contracts, (4) data model, (5) main flow, (6) errors and edge cases, (7) design decisions.
- **Controller:** receives and checks incoming input.
- **Service:** applies business rules.
- **Repository:** reads or writes stored data.
- **Operation contract:** defines inputs, outputs, and possible errors.

Example

For Session Scheduling: input = mentor, learner, and time slot; check that the slot is free; save the booking; return confirmation. If the slot is taken, return a clear error such as SLOT_TAKEN.

Common mistake

Writing only the happy path, ignoring bad input or failures, or designing an LLD that contradicts the component's stated responsibility.

Exam-ready answer

LLD specifies the internal structure and behaviour of a component. It defines operations, data models, processing flow, errors, edge cases, and design decisions so implementation and testing can proceed consistently.

Remember: What goes in? What happens inside? What comes out? What can go wrong?